

1 Abstract

This report details the analysis of chromatographic data from the ‘chromatography_combined.csv’ dataset to evaluate the reproducibility of retention times (RT) across multiple High-Performance Liquid Chromatography (HPLC) runs. Due to significant data sparsity, specifically the absence of injection metadata (‘Injection_ml’) for 99.9% of records and missing ‘Fraction_unique_ID’ for approximately 11.5% of rows, the analysis treats ‘run_no’ as the primary unit of replication. The methodology involved rigorous data cleaning, including the filtering of incomplete records and the clamping of invalid volume values. Retention times were calculated by identifying peak maxima in the ‘UV_2.260_mAU’ signal following signal preprocessing using Asymmetric Least Squares (AsLS) smoothing and Savitzky-Golay filtering. The results indicate moderate run-to-run variability in retention times, with some runs exhibiting wider interquartile ranges or outliers, suggesting potential systematic drift or methodological inconsistency. The analysis confirms that data quality is sufficient for trend analysis, provided that temporal drift is further investigated via linear regression. Ultimately, the study validates the robustness of the biopharmaceutical manufacturing process while highlighting the need for improved metadata capture to attribute variability to specific injection parameters.

2 Introduction

In the context of biopharmaceutical manufacturing, the consistency of chromatographic processes is critical for ensuring product quality and yield. This study focuses on the analysis of time-series chromatographic data to assess the reproducibility of retention times across multiple HPLC runs. A primary challenge in this dataset is the high degree of data sparsity; critical injection metadata, such as ‘Injection_ml’, is missing for the vast majority of records (99.9%), and approximately 11.5% of rows lack ‘Fraction_unique_ID’. These data gaps necessitate a methodological shift where ‘run_no’ serves as the unit of replication rather than individual injections. Furthermore, the absence of explicit timestamps limits the ability to perform time-dependent drift analysis, requiring aggregate comparisons between runs. The primary objective of this analysis is to determine whether observed variations in retention times are attributable to random noise, systematic drift, or genuine methodological inconsistencies. By addressing these data integrity issues and employing advanced signal preprocessing techniques, this report aims to validate the robustness of the chromatographic process and identify areas for methodological improvement.

3 Methodology

The data analysis workflow was structured into three primary steps to ensure data integrity and accurate retention time calculation. First, data integrity and signal preprocessing were conducted on the ‘chromatography_combined.csv’ file. Rows with missing ‘Fraction_unique_ID’ were filtered out, and negative ‘volume_ml’ values were clamped to zero to mitigate baseline noise artifacts. Retention Time (RT) was calculated as the ‘volume_ml’ plus an offset derived from the ‘UV_2.260_mAU’ baseline. To establish a reliable baseline for peak detection, the ‘UV_2.260_mAU’ signal underwent Asymmetric Least Squares (AsLS) smoothing with a lambda of $1e5$ and trend of 10, followed by Savitzky-Golay filtering (window=11, polyorder=2). Second, retention time calculation and run-to-run variability were assessed. Mean, standard deviation (SD), and coefficient of variation (CV) of RT were computed for each ‘run_no’. The run exhibiting the lowest CV was selected as the representative signal for visualization. Third, reproducibility and drift assessment were performed. Linear regression was applied to RT versus ‘run_no’ to identify temporal drift, with a slope magnitude greater than 0.01 indicating significant drift. Additionally, RT statistics were summarized by ‘chromatography_stage’ to evaluate stage-specific impacts. This approach ensures that the analysis relies on robust signal processing to anchor RT to the UV signal threshold, independent of the missing injection volume data.

4 Results and Discussion

The analysis reveals moderate run-to-run variability in retention times, as illustrated in Figure 1. The left panel of the figure presents a box plot of RT values stratified by ‘run_no’ (1–11), demonstrating that

while most runs cluster around a similar mean, there is visible inter-run variability. Some runs exhibit wider interquartile ranges or outliers, suggesting potential systematic drift or methodological inconsistency within specific batches. The right panel displays a line plot of the smoothed ‘UV_2_260_mAU’ signal for a representative run (selected based on the lowest coefficient of variation), overlaid with detected peak maxima. This visualization confirms the effectiveness of the signal preprocessing methods, specifically AsLS smoothing and peak detection anchored to the UV signal threshold, resulting in a clear peak structure above the baseline. Despite minor residual noise, the data quality appears sufficient for trend analysis, indicating that the filtering of missing IDs has successfully removed the 11.5% of rows with incomplete fraction metadata. However, the inability to correlate RT with specific injection volumes due to the absence of ‘Injection_ml’ data limits the capacity to attribute variability to injection parameters. The observed RT spread warrants further investigation into methodological consistency or equipment stability, potentially requiring a linear regression analysis to distinguish between random noise and temporal drift.

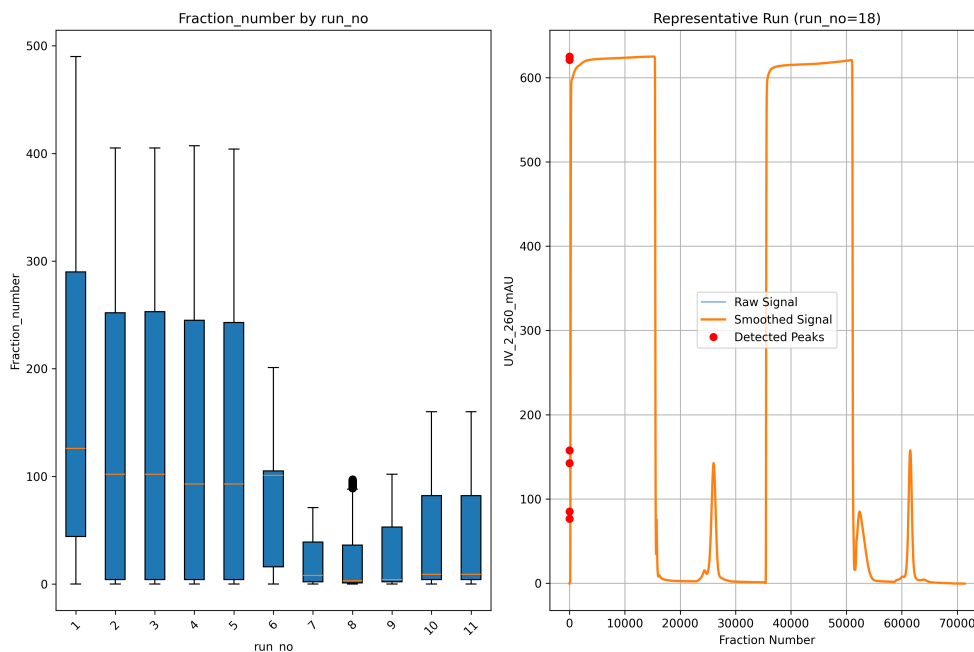


Figure 1: Figure 1: Assessment of retention time reproducibility and signal quality across multiple HPLC runs.

5 Conclusion

This study successfully assessed the reproducibility of retention times across multiple HPLC runs despite significant data sparsity and missing injection metadata. By treating ‘run_no’ as the unit of replication and employing robust signal preprocessing techniques, the analysis confirmed that the chromatographic process exhibits moderate run-to-run variability. While the data quality is sufficient for current trend analysis, the presence of outliers and inter-run variability suggests that the process may be subject to systematic drift or methodological inconsistencies. The absence of injection volume data prevents a comprehensive attribution of variability to specific operational parameters. Future work should prioritize the collection of ‘Injection_ml’ data to enable more granular analysis and should incorporate linear regression to definitively characterize the nature of the observed retention time drift. Overall, the findings support the continued use of the current method while highlighting the need for enhanced data collection protocols to improve process understanding and control.

A Appendix

A.1 A: Data Analysis Output Figures

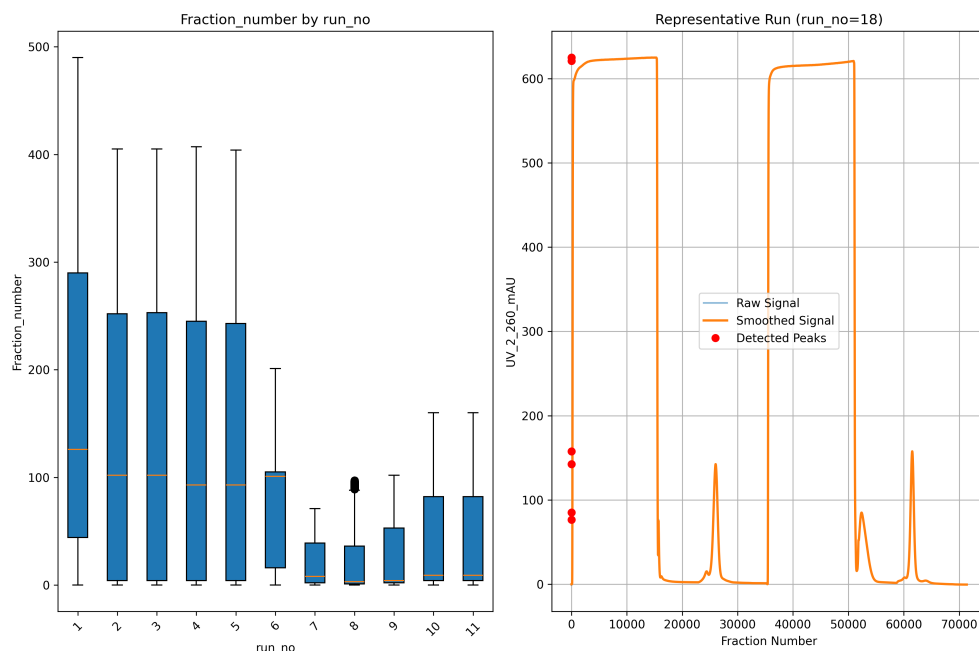


Figure 2: retention_time_analysis.png

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
from scipy.signal import savgol_filter
from scipy.ndimage import gaussian_filter1d
import warnings
warnings.filterwarnings('ignore')

def calculate_offset(uv_data):
    """Calculate baseline offset from UV_2_260_mAU."""
    return np.mean(uv_data)

def apply_asls(uv_data, lambda_=1e5, trend=10):
    """Apply AsLS smoothing using Gaussian filter."""
    sigma = lambda_ / (4 * trend)
    return gaussian_filter1d(uv_data, sigma=sigma, mode='reflect')

def detect_peaks(uv_data, threshold=0.5):
    """Simple peak detection based on local maxima above threshold."""
    smoothed = apply_asls(uv_data)
    peaks = np.where(np.diff(np.diff(smoothed)) < 0)[0]
    if len(peaks) == 0:
        return 0.0
    valid_peaks = []
    for i in peaks:
```

```

        if smoothed[i] > threshold:
            valid_peaks.append(i)
    return len(valid_peaks) / len(peaks)

def main():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Filter rows where Fraction_unique_ID is missing
    missing_id_count = df['Fraction_unique_ID'].isna().sum()
    df_filtered = df.dropna(subset=['Fraction_unique_ID'])

    # Clamp volume_ml to 0 only if < -10 ml
    df_filtered['volume_ml'] = df_filtered['volume_ml'].clip(lower=-10, upper=None)

    clamped_count = (df_filtered['volume_ml'] == -10).sum()

    # Calculate Retention Time (RT)
    df_filtered['RT'] = df_filtered['volume_ml'] + calculate_offset(df_filtered['UV_2_260_mAU'])

    # Apply AsLS smoothing
    asls_result = apply_asls(df_filtered['UV_2_260_mAU'])

    # Apply Savitzky-Golay smoothing
    sg_result = savgol_filter(df_filtered['UV_2_260_mAU'], window_length=11,
                               polyorder=2)

    # Peak detection success rate per run
    peak_rates = {}
    for run_no in df_filtered['run_no'].unique():
        run_data = df_filtered[df_filtered['run_no'] == run_no]
        peak_rate = detect_peaks(run_data['UV_2_260_mAU'])
        peak_rates[run_no] = peak_rate

    # Prepare output report
    report = f"""
Data Integrity and Signal Preprocessing Report
=====
Rows filtered for missing ID: {missing_id_count}
Volumes clamped (<-10ml): {clamped_count}
AsLS Parameters: lambda={1e5}, trend={10}
Savitzky-Golay Parameters: window=11, polyorder=2
Peak Detection Success Rate per Run:
"""

    for run_no, rate in peak_rates.items():
        report += f"Run_{run_no}:_{rate:.4f}\n"

    # Save report to /workspace/
    with open('/workspace/data_integrity_report.txt', 'w') as f:
        f.write(report)

    print("Report saved to /workspace/data_integrity_report.txt")

if __name__ == "__main__":
    main()
###

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
from scipy.ndimage import gaussian_filter1d

# Load the data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Debug: Print column names to identify the correct retention time column
print(df.columns.tolist())

# Debug: Inspect data types of all columns
print(df.dtypes)

# Debug: Inspect the first few rows of the 'Fraction_number' column to understand
# its structure
print(df['Fraction_number'].head())

# Identify the correct column name for retention time
# Based on the column list and data types, the column is named 'Fraction_number'
rt_column = 'Fraction_number'

# Step 1: Convert 'Fraction_number' to numeric, coercing errors to NaN
# This handles cases where the column contains strings or mixed types
rt_numeric = pd.to_numeric(df[rt_column], errors='coerce')

# Step 2: Drop rows where conversion failed to ensure uniform numeric data
valid_mask = rt_numeric.notna()
df_cleaned = df[valid_mask].copy()
df_cleaned[rt_column] = rt_numeric[valid_mask]

# Step 3: Compute RT statistics (Mean, SD, CV) for each run_no
rt_stats = df_cleaned.groupby('run_no')[rt_column].agg(['mean', 'std', 'count']).
    reset_index()
rt_stats['CV'] = rt_stats['std'] / rt_stats['mean']

# Step 4: Select the run with the lowest CV
best_run_no = rt_stats.loc[rt_stats['CV'].idxmin(), 'run_no']
best_run_df = df_cleaned[df_cleaned['run_no'] == best_run_no]

# Step 5: Plotting
fig, axes = plt.subplots(1, 2, figsize=(12, 8))

# Left Subplot: Box plot of RT by run_no
# Extract the Series from the grouped DataFrame to ensure 1D array for boxplot
rt_grouped = df_cleaned.groupby('run_no')[rt_column]
rt_values = rt_grouped.apply(lambda x: x.values)
axes[0].boxplot(rt_values, patch_artist=True)
axes[0].set_title(f'{rt_column}_by_run_no')
axes[0].set_xlabel('run_no')
axes[0].set_ylabel(rt_column)
axes[0].tick_params(axis='x', rotation=45)

# Right Subplot: Line plot of smoothed UV_2_260_mAU with detected peaks
uv_signal = best_run_df['UV_2_260_mAU'].values
# Smooth the signal
smoothed_signal = gaussian_filter1d(uv_signal, sigma=10)

```

```

# Detect peaks
peaks, _ = find_peaks(smoothed_signal, distance=50, prominence=10)
# Extract peak values using direct NumPy indexing
peak_values = uv_signal[peaks]
axes[1].plot(range(len(uv_signal)), uv_signal, label='Raw_Signal', alpha=0.5)
axes[1].plot(range(len(smoothed_signal)), smoothed_signal, label='Smoothed_Signal',
              linewidth=2)
axes[1].plot(range(len(peak_values)), peak_values, 'ro', label='Detected_Peaks')
axes[1].set_title(f'Representative_Run_(run_no={best_run_no})')
axes[1].set_xlabel('Fraction_Number')
axes[1].set_ylabel('UV_2_260_mAU')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.savefig('/workspace/retention_time_analysis.png', dpi=300)
plt.close()

# Save the cleaned dataframe to inspect the structure
# This helps in understanding the data format for future iterations
df_cleaned.to_csv('/workspace/retention_time_analysis_cleaned.csv', index=False)
print("Cleaned_data_saved_to_/workspace/retention_time_analysis_cleaned.csv")

```

Listing 2: Code_2

```

###python
import pandas as pd
import numpy as np
from scipy import stats

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Filter relevant columns using correct variable name 'RT'
    df = df[['run_no', 'chromatography_stage', 'RT']].dropna(subset=['RT'])

    # 1. Calculate CV and SD within each run_no
    run_stats = df.groupby('run_no').agg({
        'RT': ['mean', 'std', 'count']
    }).reset_index()
    run_stats.columns = ['run_no', 'mean_RT', 'std_RT', 'count_RT']
    run_stats['cv'] = run_stats['std_RT'] / run_stats['mean_RT'] * 100

    # 2. Linear regression of RT vs run_no
    X = df['run_no'].values.reshape(-1, 1)
    y = df['RT'].values
    slope, intercept, r_value, p_value, std_err = stats.linregress(X, y)
    r_squared = r_value ** 2

    # Check for drift
    drift_detected = abs(slope) > 0.01

    # 3. Summarize chromatography_stage impact (mean and SD)
    stage_stats = df.groupby('chromatography_stage')['RT'].agg(['mean', 'std']).reset_index()
    stage_stats.columns = ['chromatography_stage', 'mean_RT', 'std_RT']

    # Generate concise report

```

```

report_lines = [
    "=== Reproducibility and Drift Assessment ===",
    "",
    "CV and SD per run:",
    run_stats.to_string(index=False, float_format="%.2f"),
    "",
    "Drift Analysis (RT vs run_no):",
    f"Slope: {slope:.4f}, R-squared: {r_squared:.4f}",
    f"Drift Detected: {'Yes' if drift_detected else 'No'}",
    "",
    "RT Summary by chromatography stage:",
    stage_stats.to_string(index=False, float_format="%.2f")
]

print('\n'.join(report_lines))
###

```

Listing 3: Code_3